The Binary You Ship Is the Binary You Test

Massimiliano · OpenSouthCode 2026

`$whoami`

- Massimiliano Giovagnoli
- Software engineer @ Chainguard
- OSS, building and observing all the things
- Music, racing cars, nature
- `@maxgio92` on GitHub, X and Telegram, `@maxgio92.bsky.social`, `@maxgio92@hachyderm.io`
- linkedin.com/in/maxgio

Quality

- How do you ensure that your software works?
- How do you ensure that your strategy is telling you a reliable story?
- How do you ensure that positive signals are meaningful?

The standard coverage story

Coverage today: the build-time approach

```
# Go
go build -cover -covermode=count -coverpkg=./... -o myapp ./main.go

# C/C++ with GCC
gcc -fprofile-arcs -ftest-coverage -o myapp src/*.c

# LLVM source-based
clang -fprofile-instr-generate -fcoverage-mapping -o myapp src/*.c
```

- Instrumentation is **injected at compile time**
- Produces a separate artifact: the instrumented build
- Run tests against that artifact, collect the report

Testing Linux distro packages

```yaml
- name: crane-cov
  description: "Crane compiled for collecting coverage profiles"
  pipeline:
    - uses: go/build
      with:
        extra-args: -cover
  test:
    environment:
      contents:
        packages:
          - busybox
          - go
      environment:
        GOCOVERDIR: /home/build
    pipeline:
      - name: Run a command with the instrumented binary
        runs: |
          crane manifest chainguard/static
      - name: Report function coverage
        runs: |
          go tool covdata func -i=.
```

The problem at scale

```
package-foo/
  build/
    package-foo-1.0.0.apk                ← what you ship
    package-foo-1.0.0-instrumented.apk   ← what you test
```

- Every package needs two build targets
- You maintain **thousands of packages** (Go, C, C++, Rust, Python extensions…)
- Doubled CI time, doubled storage, doubled maintenance
- Reproducibility: **You're not testing what you ship**

What if coverage didn't require a separate build at all?

The kernel sees everything

- **uprobes**: Linux kernel user-level dynamic tracing
- Available since Linux 3.5 [1]
- Attach to a function by **binary path + offset**
- Works on **any ELF binary**
- Integrated into eBPF since Linux 3.18 [2] - `BPF_PROG_TYPE_UPROBE`

> 1. *https://lwn.net/Articles/499190/*
> 2. *https://lwn.net/Articles/637391/*

What is a uprobe?

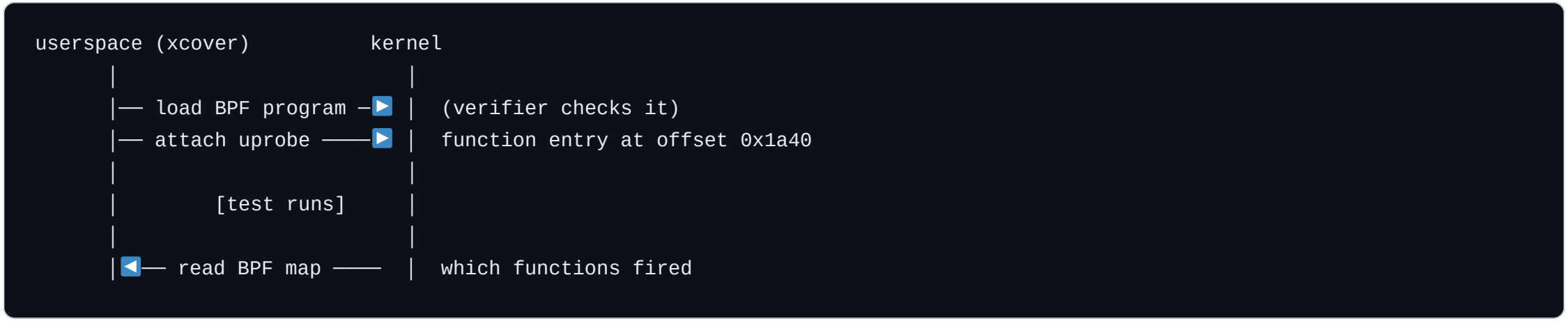
```
Loaded image:
  offset 0x1a40: PUSH RBP        ← function entry
  offset 0x1a41: MOV RBP, RSP

With uprobe attached:
  offset 0x1a40: INT3            ← kernel patches this
                 (original bytes saved)
```

At runtime, on each call:

1. Software interrupt causes a trap into kernel mode
2. BPF program runs
3. Registers restored, execution continues in userland

eBPF makes uprobes programmable

- **eBPF**: run sandboxed programs in the kernel, triggered by events
- Loaded programs are **verifier-checked**: no crashes, bounded execution, safe
- **Attach** programs to specific kernel paths - i.e. uprobe
- Access to kernel data and communicate with userland with **maps**



Introducing xcover

> *The binary you ship is the binary you test.*

- eBPF uprobe-based coverage profiler for compiled binaries
- Runs as a **daemon** alongside your test suite
- Attaches to the binary at its path
- Reports function-level coverage when you stop it
- Ships as a **single static binary**, zero runtime dependencies

`github.com/maxgio92/xcover`

xcover in practice

Architecture

xcover API

```
tracee := trace.NewUserTracee(
    trace.WithTraceeExePath("./myapp"),
    trace.WithTraceeSymPatternInclude(`^github\.com/myorg`),
    trace.WithTraceeSymPatternExclude(`_test$`),
)

tracer := trace.NewUserTracer(
    trace.WithTracerLogger(logger),
    trace.WithTracerReport(true),
    trace.WithTracerTracee(tracee),
)

if err := tracer.Init(ctx); err != nil { ... }
if err := tracer.Run(ctx); err != nil { ... }
```

The CLI workflow

```
# Start the profiler (detach to background)
xcover run --path ./myapp --detach

# Wait until all probes are attached (ready to trace)
xcover wait

# Run your tests — nothing changes here
./myapp foo
./myapp bar
./myapp baz

# Stop the profiler and collect the report
xcover stop
```

Filtering functions

```
# Only trace functions in your own packages
xcover run --path ./myapp \
  --include '^github\.com/myorg/myapp' \
  --detach

# Exclude generated code
xcover run --path ./myapp \
  --include '^github\.com/myorg' \
  --exclude '\.pb\.go' \
  --detach
```

- `--include` and `--exclude` take Go regexes
- Applied to function names at probe-attachment time

The coverage report

```
{
  "exe_path": "./myapp",
  "funcs_traced": [
    "github.com/myorg/myapp/pkg/server.HandleRequest",
    "github.com/myorg/myapp/pkg/server.parseConfig",
    "github.com/myorg/myapp/pkg/db.Connect",
    ...
  ],
  "funcs_ack": [
    "github.com/myorg/myapp/pkg/server.HandleRequest",
    "github.com/myorg/myapp/pkg/db.Connect"
  ],
  "cov_by_func": 0.72
}
```

- `funcs_traced`: all functions with probes attached
- `funcs_ack`: functions called at least once during the run
- `cov_by_func`: ratio: 72% of functions were exercised

Demo 

One catch:
stripped binaries

What strip actually removes

```
$ ls -lh myapp myapp-stripped
-rwxr-xr-x 1 user user 14M myapp
-rwxr-xr-x 1 user user  6M myapp-stripped

$ readelf --sections myapp | grep -E '\.symtab|\.debug'
  [32] .symtab           SYMTAB  ...  ← function names + offsets
  [33] .strtab           STRTAB  ...  ← symbol name strings
  [34] .debug_info       PROGBITS ...
  [35] .debug_line       PROGBITS ...

$ readelf --sections myapp-stripped | grep -E '\.symtab|\.debug'
$
$ readelf --symbols myapp-stripped
$
```

- `strip` removes `.symtab`, `.strtab`, and all `.debug_*` sections
- **The code is still there.** Only the metadata is gone.

What strip does NOT remove

```
$ readelf --sections myapp-stripped | grep '\.eh_frame'
  [17] .eh_frame_hdr     PROGBITS ...
  [18] .eh_frame         PROGBITS ...
```

- `.eh_frame`: DWARF Call Frame Information, needed for stack unwinding
- Survives `strip --strip-all`: needed for **exception handling** (e.g. C++)
- Encodes function boundaries and stack layout
- The compiler always emits it. The linker always keeps it.

Introducing resurgo library

Static function recovery for stripped ELF binaries.

Deterministic:

1. **DWARF CFI** (`.eh_frame` ELF section): function ranges, survives strip, high confidence

Heuristic:

1. **Prologue patterns**: at function entry, architecture-specific
2. **Call-site analysis**: `call` targets are function entry points by definition
3. **Alignment boundaries**: compilers align functions to 16/32-byte boundaries

Heuristic- and determinism-based validation for confidence.

`github.com/maxgio92/resurgo`

xcover function recovery with resurgo

Transparent to the user

```
# Unstripped binary
$ xcover run --path ./myapp --detach --log-level debug
DBG  resolved 1842 functions via .symtab

# Stripped binary — same command
$ xcover run --path ./myapp-stripped --detach --log-level debug
DBG  .symtab not found, falling back to static recovery
DBG  resolved 1791 functions via resurgo (eh_frame + prologue analysis)
```

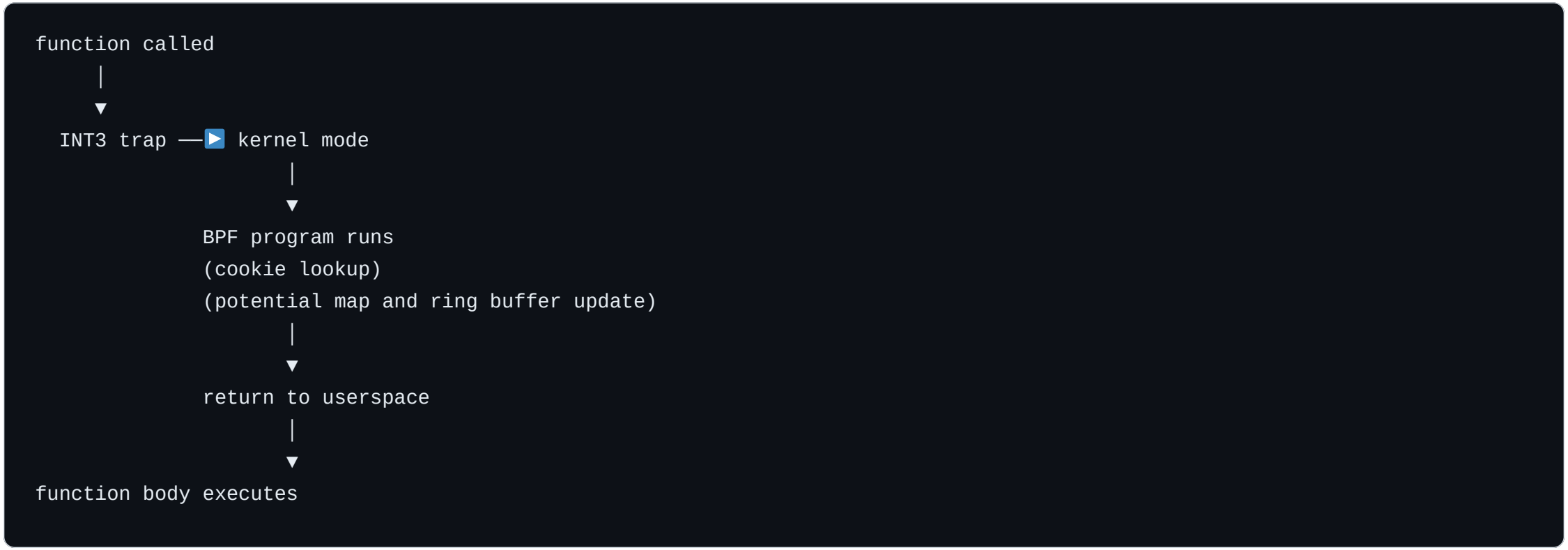
- xcover detects the binary is stripped and calls resurgo automatically
- No flags, no config change, no separate step
- Coverage report is identical in format

Demo 

But what's the cost?

The overhead model

Every call to a traced function pays:



- **Fixed cost per call**, independent of function body size
- Overhead scales with **call frequency**, not binary size or number of functions

The probe

```
SEC("uprobe/handle_user_function")
int handle_user_function(struct pt_regs *ctx) {
        __u64 cookie = bpf_get_attach_cookie(ctx);
        u8 seen = 1;

        /* Check if the function has been already reported */
        if (bpf_map_lookup_elem(&seen_funcs, &cookie)) {
                return 0;
        }

        /* Track which functions have been reported */
        bpf_map_update_elem(&seen_funcs, &cookie, &seen, BPF_ANY);

        struct event_t *event = bpf_ringbuf_reserve(&events, sizeof(struct event_t), 0);
        if (!event) {
                return 0;
        }

        event->cookie = cookie;
        bpf_ringbuf_submit(event, ringbuffer_flags);

        return 0;
}
```

Benchmark setup

Scenario	Description	How
Baseline	No tracing	Tracee runs without uprobes
Idle	uprobe attached	Probe attached to functions that never run
Hit	uprobe firing, already seen	Tracee runs the same probed function N times
Miss	uprobe firing, new function	Tracee runs N probed unique functions only once

Aggregate overhead

Scenario	Description	Time per call	Overhead vs Baseline
Baseline	No tracing	~2 ns	
Idle	uprobe attached	~2 ns	
Hit	uprobe firing, already seen	~2000 ns	1000x
Miss	uprobe firing, new function	~4000 ns	2000x

> ***+X% wall time for full test coverage on the exact production binary.***

- Filtering to your own packages reduces overhead
- Overhead is deterministic; no jitter, no GC interaction
- Acceptable for CI pipelines; not for latency-sensitive benchmarks

Memory: BPF maps

```
Map: per-function event counter
  Key size:   8 bytes (cookie / function ID)
  Value size: 8 bytes (uint64 hit count)
  Max entries: N (one per traced function)

Total: N × 16 bytes
```

- 1000 functions → 16 KB
- 10,000 functions → 160 KB
- **Negligible** compared to typical process memory

The tradeoff

	Instrumented build	xcover
Build changes required	Yes	**No**
Works on stripped binaries	No	**Yes**
Cross-language	No	**Yes**
Same binary as production	No	**Yes**
Per-call overhead	~0	~2000 ns
Setup complexity	Per-language	Once

> *You pay a small, predictable cost per function call.*
> *In exchange: no instrumented builds, one tool, any binary.*

Eliminate the kernel trap

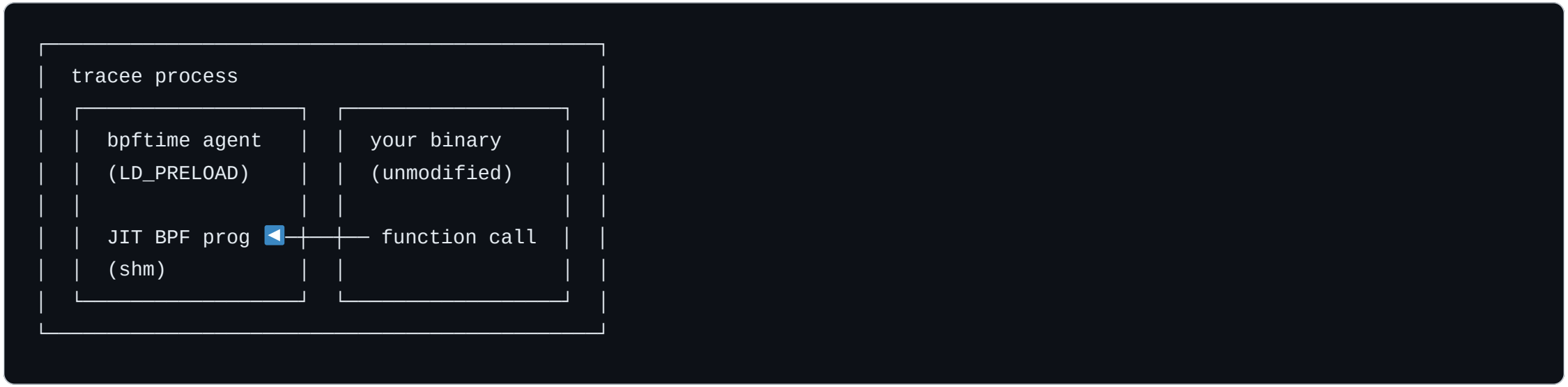
The cost is in the trap

```
Current: kernel uprobe
  function call → INT3 → kernel trap → BPF runs → return to userspace

Hypothesis: userspace BPF
  function call → intercept in-process → BPF runs → continue
                  (no kernel transition)
```

- Userspace eBPF runtimes: run BPF programs in the same process as the tracee
- No kernel involved → no trap → potentially zero context-switch overhead
- The BPF program runs as a JIT-compiled function in userspace

eunomia-bpf/bpftime

- Userspace eBPF runtime: `syscall_server.so` and `agent.so` libraries
- `syscall_server` intercepts `perf_event_open` / `bpf_link_create` syscalls via `LD_PRELOAD`
- `syscall_server` creates data structures in a shared memory, read by the `agent`
- `agent` sets up the trampolines at function entry to the BPF `BPF_PROG_TYPE_UPROBE` program
- No kernel trap
- API-compatible with kernel eBPF



xcover + bpftime: the experimental path

```
# Start the bpftime syscall server (intercepts BPF syscalls from xcover)
xcover run --path ./myapp --userspace-bpf --detach

# Inject the bpftime agent into the tracee
LD_PRELOAD=$(xcover agent extract) ./myapp
```

- `xcover agent extract` unpacks the embedded bpftime shared library
- bpftime `agent` intercepts uprobe hits in-process
- Everything else is identical: same report, same workflow
- Flag: `--userspace-bpf` (experimental)

Let's benchmark again!

Scenario	Description	Overhead reduction Userspace vs Kernel
Baseline	No tracing	~
Idle	uprobe attached	~
Hit	uprobe firing, already seen	**-53%**
Miss	uprobe firing, new function	**-54%**

Current status of the userspace mode

What works

- Single-uprobe attachment (perf-event based) ✅
- bpf_cookie propagation (function identification) ✅
- Coverage report generation ✅

Known limitations

- No `uprobe_multi` link support (perf-based)
- Requires `LD_PRELOAD` injection (not transparent for `execve`-started processes)
- Statically linked / musl binaries: no agent injection ⚠️
- Frida Gum interceptor: some aggressive compiler optimisations (tail-call elision, LTO) are not yet handled

> *If you've worked with userspace eBPF runtimes, find me after the talk.*

Limitations & what's next

Limitations

- Function inlining defeats uprobe-based interception
- There is an overhead cost
- Userspace mode requires dynamically linked tracee (`LD_PRELOAD`ed agent)
- Frida Gum interceptor: some aggressive compiler optimisations (tail-call elision, LTO) are not yet handled

Upstream gaps

- **bpftime**: some patches that are going to be proposed upstream
- **libbpfgo**: missing `AttachUprobeWithCookie` libbpf API (single-uprobe attach with per-probe `bpf_cookie`)
  - https://github.com/aquasecurity/libbpfgo/pull/523

What I'm working on

- Rolling out xcover on thousands of packages
- Performance optimizations
- Stress test the userspace mode

Wrapping up

The binary you ship is the binary you test.

- Coverage tooling today assumes you control the build. At scale, that assumption breaks
- Kernel instrumentation allows to observe the runtime
- The overhead is real, measured. It can be acceptable for test workloads
- If you want to cut the overhead, userspace BPF runtimes can reduce it up to 4x

`github.com/maxgio92/xcover` · `github.com/maxgio92/resurgo`

Questions?

linkedin.com/in/maxgio
@maxgio92 on Telegram

github.com/maxgio92/xcover
github.com/maxgio92/resurgo